Qontext: a Quipu-inspired Context Memory

A quipu-inspired conversational memory: what it saves, when it helps, and where it stops helping

Hylke Siegersma, with Claude • July 2026

Summary: what is established, and under what conditions

This report was written as the work happened, so it argues with itself in places. Later sections correct earlier ones on the strength of better measurements, and where two passages disagree **the later one stands**. This summary states the position the evidence actually supports, so no reader has to reconstruct it from the chronology.

1. The cost result, which holds everywhere it was tested. A Qontext pack answers the same questions as a full transcript while sending a small fraction of the tokens. At 26 turns the pack costs about 17% of the transcript; at 800 turns, about 1%, because the transcript grows and the pack does not. Replicated across three seeds, four distractor densities and two model families, with accuracy tied at 10/10 in every cell of the larger model's sweep. This is the claim that needs no conditions.

2. The accuracy result, which is real but conditional. On a 4B and on a 9B model, a pack does not merely match a full transcript — it beats it, by up to three points out of ten, once the transcript contains plausible wrong answers. On a 12B the effect vanishes entirely: that model scores 10/10 from the transcript at every distractor density tested. The honest statement is therefore conditional: **context reduction improves accuracy when the context holds plausible wrong answers and the model is at or past its ability to choose among them.** Distractor density sets the difficulty; model capability sets the threshold. Neither alone predicts the effect.

3. The library. qontext_memory.py is a single-file, dependency-free Python memory with 91 tests, plus a roleplay build and a SillyTavern port checked for parity against it. Section 8 gives the three design rules that survived the runs.

The main limitations, stated once here rather than buried. The ten planted facts and the ten questions used throughout are ours. The conversation surrounding them is not: the final runs replace the templated filler with 30,000 human-written exchanges from DailyDialog, and the headline result below is measured there. Earlier sections were measured on fully synthetic conversations and are marked where that matters. Everything comes from one machine and two local quantised models, and answers are scored by keyword match rather than by judgement.

1. The two questions

This study began with one idea and ended up answering two questions. The idea: the Inca quipu recorded information as knots on cords — not a transcript of everything said, but a deliberate selection of what was worth knotting. Applied to AI, could a conversation memory that stores few, dense, self-contained entries ("Qontext") carry the same meaning as a full transcript in far less text?

The questions that emerged: **(1)** Can a small local model (Gemma 4 E4B, ~4B parameters, running on llama.cpp) build such a structure autonomously, given tools and a long turn budget? **(2)** Is the Qontext structure itself actually effective — more meaning in less text?

2. Setup

A ~300-line Python harness drives the model as an autonomous agent against a local llama-server. The model gets a task prompt and six text-based tools (list, read, write, append, run, test) expressed as fenced blocks — chosen over JSON tool-calling because small models imitate text formats more reliably. Every run logs each turn to run_log.jsonl; a deterministic acceptance test scores the result. Turn budget: 60. All inference fully local.

Model note. Findings F1–F5 and every harness run used the Gemma 4 E4B GGUF. The scaffold comparison in §6 used a different model — Shirdel-Coder-9B-Claude-Fable-5.Q6_K, a 9B — because the intended coder GGUF proved unusable (Appendix B) and the exact E4B GGUF from the harness runs was no longer on hand, so a matched pair was not available. Where this report says "E4B" it refers to the harness runs and the benchmark; §6 names its model explicitly.

3. The four autonomy runs

Run	Task / scaffold change	Parse failures	Result	Failure mode
1	Byte-compression memory; bare scaffold	21 of 60	Fail	Cut-off writes; 17× repeat loop
2	Semantic memory task; example reply, append tool, loop breaker added	0 of 60	Fail	Whack-a-mole between density and relevance
3	Same task; worked example added to prompt	0 of 60	PASS (turn 31)	—
4	Improve own solution to generalize; held-out test set	0 of 60	Fail (close)	Seesaw: any two of three constraints, never all three

Run 1 → 2: fixing the interaction layer

Run 1 wasted a third of its budget on malformed tool blocks — mostly 5,000-character file rewrites cut off mid-generation — and spent turns 8–60 rewriting the same file against the same failure, 17 times. Three scaffold changes followed: a concrete example reply in the system prompt, an append tool so files can grow in small pieces, and a loop breaker that injects a hint after three identical failures. Run 2: zero parse failures, all 60 turns productive, 3 hours faster. No model change — only scaffold.

Run 2 → 3: the worked example

Run 2 still failed: told to keep stored text under half of observed, it truncated entries into meaningless fragments (". Anyway", "Rain again") rather than storing fewer, better ones. Run 3 added a worked example — one input turn, two good entries, four labelled bad ones. The model then found the strategy, used a self-initiated debug script, and passed at turn 31. Rules told it what to do; the example showed it, and only the example worked.

Run 4: the capability ceiling

The final run asked the model to make its own passing solution generalize: a visible conversation A, a held-out conversation B with different phrasings, and a density limit. The log shows a clean seesaw — a lean version (density 0.35, strong on A, weak on B) alternating with a greedy version (density 0.75, stronger on B, over budget). It improved from 5/10 to 8/10 on A and 5/10 to 7/10 on B, adopted the "name the subject" rule, but never found the move that satisfies all three constraints at once: better extraction precision. A hand-built implementation proves that move exists. Sixty turns of sliding along a trade-off curve, no jump to a better one.

4. The Qontext benchmark

The structure itself was tested by having the same E4B model answer ten factual questions about an unseen 26-turn conversation under four conditions: the full raw transcript, the pack built by E4B's own memory implementation, the pack built by a hand-built ("ceiling") implementation, and no context at all. Questions were retried at rising temperature (0.2 → 0.7 → 1.0); a question counts as correct if any try contains the expected answer.

Condition	Accuracy	Tokens per call	Note
Full transcript	4 / 10	477	Six questions answered empty — the model drowned
Qontext (ceiling)	10 / 10	80	Every answer correct at 17% of the token cost
Qontext (E4B's)	2 / 10	130	Fragments without payloads; ranking collisions

No context	0 / 10	53	Floor: keyword guessing yields nothing

The headline of this first benchmark was not that the pack is cheaper but that the full transcript was **worse**: a 4B model given more text answered less, and the dense pack was the only condition with every answer correct. That result is real and it reproduced later on a 9B under distractors — but it does not hold on every model, and §7 documents a 12B for which it disappears. The cheapness, by contrast, held in every condition ever measured.

5. Findings

F1 — Scaffold quality dominates. The same model went from 21 wasted turns to zero, and from looping to self-directed debugging, purely through scaffold changes: an example reply, an append tool, a loop breaker, error messages that say what to do next.

F2 — Worked examples beat rules. Every behavioral breakthrough in this study (tool format, extraction strategy) came from showing one concrete example, not from adding instructions.

F3 — Context reduction improves accuracy when the context holds plausible wrong answers and the model is at or past its ability to choose among them. Measured on Gemma E4B as 4/10 from a transcript against 10/10 from a pack, and reproduced on a 9B reasoning model at 800 turns, where 60% distractor density took the transcript to 4.3/10 against the pack's 8.0. It did not reproduce on a 12B, which scored 10/10 from the transcript at every density. This is the fourth and narrowest statement of the finding; §7 gives the counterexample in full.

F4 — The Qontext idea works, and its saving grows with the conversation. A quipu-style selection of dense, self-contained entries preserved 10/10 answerable facts at 17% of the tokens on a 26-turn conversation. The ratio is not fixed: a pack is flat in size while a transcript grows, so the same comparison at 800 turns is roughly 1% — 67 to 100 tokens against 8,000 to 9,000. Where the two figures differ in this report, they are measuring conversations of different lengths, not disagreeing.

F5 — Extraction generalization is the small-model frontier. E4B operated tools flawlessly for 60-turn stretches, held a goal, debugged itself — and still could not lift extraction precision to pass a held-out set. It optimizes within a strategy; it does not yet invent the better strategy.

F6 — A frontier scaffold assumes context is free. Claude Code's opening prompt is ~27K tokens against the harness's ~2K, a ratio of about 13:1. Built for a 200K window, it never compacts; run against a local server it exhausted 65K without finishing a task the harness finished inside 8K. On a small local model, context discipline is not an optimisation — it is a precondition for the agent running at all. See §6.

F7 — The cost result is the one that survives every model. Across two model families, four distractor densities and three seeds, a pack of 67 to 100 tokens answered as well as a transcript of 7,900 to 9,000

— a 90x to 135x reduction with accuracy tied at 10/10 on the larger model. Where the accuracy advantage is conditional on the model, the saving is not. A technique that costs one percent of the context and gives up nothing needs no accuracy story to be worth deploying.

6. Scaffold economics: a frontier scaffold on a local model

What we set out to do

The findings above compare memory strategies while holding the scaffold fixed: the same custom harness feeds a small model either a full transcript, a Qontext pack, or nothing. The obvious next question is the mirror image — hold the model and the task fixed, and vary the scaffold. Run the same semantic-memory task under (a) our harness and (b) Claude Code, a production agent scaffold built for frontier models, and see what the scaffold itself costs.

What happened

The comparison did not produce a completion time, because Claude Code never completed. It filled the server's context window and aborted.

The cause is not subtle. Claude Code's opening prompt — system prompt, tool definitions, environment preamble, CLAUDE.md — is roughly 27,000 tokens before the model has read a single line of the task. Our harness's opening prompt is roughly 2,000 tokens, a ratio of about 13:1. Claude Code is built against a 200K-token window, so from its point of view 27K is a rounding error and there is no reason to compact early; history simply accumulates. Run against a local llama.cpp server the accumulation is terminal. We raised the server to -c 65536; that bought turns, not a finish. Raising it further trades one wall for another: the KV cache grows linearly with context, and on a 9B at Q6_K every additional turn is slower than the last. The run dies either of context exhaustion or of time.

Why this is a result and not a failed experiment

It is tempting to call this a setup problem and go looking for a configuration that lets the comparison run. That would be measuring the wrong thing. The inability to run is the measurement.

A scaffold encodes an assumption about what is scarce. Claude Code assumes context is abundant and model calls are the expensive part, so it spends context freely to save turns: exhaustive tool schemas, verbatim file contents, full history retained. That is the correct trade against a 200K window and a frontier model. Invert the assumption — a 9B model on consumer hardware, where context is the binding constraint and every token is also latency — and the same design becomes self-defeating. The scaffold spends its budget on generality before the task begins.

This closes the argument the earlier findings open. F3 and F4 showed that a dense pack can match a full transcript at a fraction of the tokens, and — on models small enough to be confused by their own context — beat it outright. Context reduction is therefore not merely an economy: at minimum it is a

very large saving for no measurable loss, and at best it is an accuracy gain as well. A scaffold that assumes context is free forecloses both, which is what the Claude Code comparison demonstrates below.

The comparison we can honestly report

	Custom harness	**Claude Code**
Opening prompt	~2K tokens	~27K tokens
Context growth	bounded (keep_last_messages trimming)	unbounded until server limit
Auto-compaction	n/a — trimming is the design	assumes 200K window; never triggers
Server context needed	8K sufficient	65K insufficient
Outcome on task	PASS at turn 31	aborted, task incomplete
Model	Gemma 4 E4B	Shirdel-Coder-9B — see §2

The turn counts are not directly comparable and we do not claim they are: the models differ, and an aborted run has no turn count to compare against. What is comparable, and what matters, is the qualitative outcome — one scaffold finished the task inside an 8K window, the other could not finish it inside 65K.

Caveat

This is a single task under a single configuration, and Claude Code was not designed for this deployment — using it here is deliberately off-label. A fairer quantitative run is available if wanted: shrink the scaffold's footprint (trim CLAUDE.md, disable unused tools) until the opening prompt fits, and report turns-completed. We expect that to narrow the gap without closing it, because the trimming is exactly the economy the scaffold otherwise declines to make — which is the finding restated, not a refutation of it.

What this changes about the study's claim

The study's original claim was about memory representation: a knotted, selective structure carries more meaning per token than a transcript. F6 widens that to a claim about scaffold economics. On small local models, context discipline is not an optimisation layer bolted on at the end; it is a precondition for the agent running at all. Qontext is one way to supply that discipline.

7. The live re-run: a model in the loop

Every number in §4 is a containment measurement — whether the answer appears in the pack — scored by keyword match rather than by asking a model. That is an upper bound on what a model could

answer, not evidence that one did. In July 2026 the benchmark was re-run end to end against a live llama.cpp server on GPU, with the shipped library added as a fifth condition alongside the original four.

Condition	Accuracy	Prompt tokens per call	Share of full
Full transcript	10/10	476	—
Model-built memory	5/10	128	27%
Hand-built ceiling	10/10	76	16%
Qontext library	10/10	69	14%
No context	1/10	52	—

What it confirms

The containment proxy was accurate: it predicted 10/10 for both the ceiling and the library, and that is what a model produced. The library matches the hand-built ceiling on accuracy while sending fewer tokens (69 against 76) from a denser memory (0.30 against 0.41). The model-built memory remains substantially worse at 5/10, which supports F5: extraction generalisation, not tool operation, is where a small model falls down.

What it contradicts

The full transcript scored 10/10. F3's inversion — 4/10 from the transcript against 10/10 from the pack — did not reproduce. The difference is the model: the original result used Gemma E4B, roughly 4B parameters and no reasoning step, while this run used a 9B reasoning model that handles a 1,566-character transcript without difficulty. The accuracy argument for context reduction is therefore narrower than this report originally implied, and belongs to small non-reasoning models.

What survives, and why it matters more

Equal accuracy at 14% of the context cost. The value is not the ratio at any one turn but the shape of both curves: a transcript grows linearly with the conversation while a pack is bounded by its budget. That was measured directly — see the table below — by planting ten facts in the opening turns, burying them under hundreds of turns of unrelated chatter, and then asking about them.

Turns	Transcript tokens	Pack tokens	Transcript accuracy	Pack accuracy
26	304	65	4/5	5/5
100	1,114	65	4/5	5/5
400	4,516	65	5/5	4/5
800	9,081	65	5/5	5/5

The pack is flat at 65 tokens from 26 turns to 800, while the transcript grows thirtyfold to 9,081 — and accuracy holds on both sides, so the saving costs nothing measurable. Offline the pack stays flat to 1,600 turns, where the transcript reaches roughly 15,000 tokens; at that point the memory has hit its 500-knot ceiling, evicted some eight hundred filler knots, and kept all ten planted facts.

A correction on latency

An earlier draft of this section claimed the transcript would cost about 90 seconds of prompt processing at 500 turns. That was wrong by roughly thirtyfold. It extrapolated from a 39-token request whose timing was dominated by fixed overhead, giving an apparent 100 tokens per second. Measured on real prompts, throughput rises with size as that overhead amortises — 434 tokens per second at 304 tokens, 3,363 at 9,081 — so the 800-turn transcript costs 2.7 seconds against the pack's 0.1. A 27-fold difference, but small in absolute terms at this scale.

What survives is the token count and the wall it eventually meets. Nine thousand tokens per call is thirty times the KV cache, thirty times the cost on any metered API, and past the point where an 8K context window has already failed. The binding constraint on a local model is the window, not the clock — which is the same conclusion §6 reached from the other direction.

Stated plainly, and not claimed beyond it: this is a technique for local models, long sessions and constrained context. Against a frontier model with a 200K window and cheap tokens, sending the transcript is simpler and better.

This was the third statement of F3, and at the time the narrowest: the original claim was too broad, the first correction too narrow, and the version that survived contact with the data was about distractor density rather than model size. A fourth statement became necessary later, when a 12B model showed the effect disappearing entirely — see §7. Read this section as the argument at that stage of the work, not as the final position.

The control makes the attribution clean, and three seeds replicate it: at 800 turns with no decoys the transcript scores 9.3/10, and with 60% decoys it averages 4.3/10 across seeds, while the pack goes from 10/10 to 8/10. Same length, same model, same questions. Length is not what breaks it; a transcript full of near-miss answers is.

Turns	Transcript accuracy	Transcript tokens	Pack accuracy	Pack tokens
400	4/10	3,989	9/10	91
800	4/10	8,042	9/10	100

The measurements above use filler that cannot be mistaken for an answer, which makes retrieval easy and flatters the pack only on cost. Replacing most of that filler with facts of the same shape about other people — another person's dog, another team's project, another day's standup — changes the result completely.

The decoy run: where the transcript actually fails

The transcript fails in three distinct ways, and which one appears depends on how crowded the context is. At low density the model commits to a decoy — answering “fox-signal” for a project codenamed heron-nest, or “September 12th” for a report due in March. At higher density it answers “unknown”, or produces nothing at all after spending its entire generation budget deliberating. The pack removes the choice rather than improving it: retrieval discards the rivals before the model sees them.

Confusable share of filler	Transcript (3 seeds)	Pack (3 seeds)
0%	9.3 (9–10)	10.0 (10–10)
10%	6.7 (4–9)	8.0 (7–9)
30%	3.7 (3–4)	6.7 (6–7)
60%	4.3 (3–6)	8.0 (6–10)

The degradation is a slope, not a cliff, and the pack wins at every density by a margin that grows with it: +0.7, +1.3, +3.0, +3.7 out of ten. Mean of three seeds at 800 turns, ten questions each, every cell measured under identical settings.

An earlier version of this table reported a collapse to 5/10 at 10% decoys and 2/10 at 30%, and called it a cliff. Those numbers were measured through a generation cap of 512 tokens, below what this model needs to finish reasoning about a crowded context. The condition that deliberates is the condition that gets truncated, so the damage fell almost entirely on the transcript. Re-measured at 4,096 tokens, the same seed and the same questions score 9/10 rather than 5/10. **Roughly half of the reported failure was the measuring instrument.**

The objection that the larger cap still binds was tested directly. Tripling it again to 12,288 tokens at 30% decoys moved transcript accuracy from 3.7 to 4.3 out of ten while nearly tripling the reasoning generated, with individual questions running to 37,699 characters of deliberation and still producing no answer. **The model does not converge given more room; it deliberates further.** The transcript’s failure is therefore a property of the context, not of the harness. What the extra budget changes is the shape of the failure: wrong answers fall from three to one while empty ones rise from eleven to sixteen.

A second cost, never previously measured, appears in the same runs. Crowding the context inflates the **generated** tokens, which are far more expensive than prompt tokens: reasoning grows 4.6-fold from 1,166 to 5,358 characters as decoy density rises, while the pack’s stays flat and at 60% decoys is eight times smaller. Every cost argument earlier in this report counted prompt tokens only, and therefore understated the difference.

One limit belongs on the record beside the result. The pack is not immune either — it falls from 10.0 to 6.7 at 30% decoys, and earlier passages implying otherwise were quoting only the favourable cells. Its losses, however, are not retrieval failures: containment was verified offline at all four densities at 30/30,

with the correct knot ranked first. The pack loses when the model hedges or exhausts its budget while holding the right answer on the first line.

A second model, and the accuracy claim does not reproduce

Every measurement above comes from one 9B reasoning model. Repeating the whole sweep against a second — a 12B instruct model at Q8_0, same machine, same conversations, same questions, three seeds per cell — produces a result this report has to absorb rather than explain away: **the larger model scores 10/10 at every distractor density, from the transcript.** One question of a hundred and twenty was missed, and that was a deliberation collapse rather than a wrong answer. The cliff, the slope, the seduction and camouflage modes — none of them appear. It reads an 800-turn transcript holding forty plausible wrong answers and picks the right one.

F3 was already on its third statement. It now has a clean counterexample, and the fourth version has to name the variable that was missing: context reduction is an accuracy win when the context contains plausible wrong answers **and the model is at or past its ability to choose among them**. Distractor density sets the difficulty; model capability sets the threshold; neither alone predicts the inversion. What was not established is why — the two models differ in size, family, training and quantisation simultaneously, so “larger models are immune” is an untested guess, not a finding.

What survives is the cost result, and on this model it is the whole claim. Identical accuracy, ten out of ten against ten out of ten, at 67 to 99 tokens against 7,900 to 9,000: **a 90x to 135x reduction in prompt tokens for no measurable loss.** A technique that costs one percent of the context and gives up nothing does not need an accuracy story to be worth deploying. The reasoning cost reproduces too — crowding inflates the larger model's generated tokens 4.5-fold, from 425 to 1,918 characters, while the pack's grows far less — and generated tokens are the expensive kind, so a real cost gap remains even where accuracy is tied.

Stated plainly, because the distinction matters more than the headline: on the 9B, Qontext is an accuracy win. On the 12B it is a hundredfold cost saving at parity. Both are worth having, they are different claims, and nothing in this report should be read as suggesting the first generalises.

The result on real conversation

The strongest objection to everything above is that our own script wrote the conversations. The final runs remove that: the filler around the planted facts is 30,000 consecutive human-written exchanges from DailyDialog, and 550 of them were discarded for containing one of our answer keywords — “Friday” occurs 226 times in the corpus, and a transcript could otherwise have matched a keyword by coincidence with the scorer unable to tell the difference.

At 800 turns on the 12B, across three seeds: accuracy 9.3 out of ten from the full transcript against 9.7 from the pack — tied, since the ranges overlap and three seeds cannot separate them — at **13,853 prompt tokens against 116, a 119-fold reduction**, with 1,697 characters of generated reasoning against

789 and 6.6 seconds of prompt processing against 0.4. The saving is the result; the accuracy is a tie, and a tie is what the cost claim needs.

One construction error is worth recording because it was caught before it reached a conclusion. Splicing unrelated dialogues under a single speaker does something no real conversation does: 13.6% of exchanges make a first-person identity claim, so the extractor produced knots reading “the user is Greg Wu, Head of Consultancy” alongside the planted “the user works as a nurse”. Those do not compete with the planted facts, they contradict them. Filtering them restored offline containment from 9/10 to 10/10 on every seed. **Real conversation costs about half a point of accuracy; the construction error cost another 0.7.** An earlier draft attributed the whole gap to realism, which was half true and half our own bug.

What survives that correction is still worth stating: real human conversation at zero decoys is a harder environment (9.3) than our engineered decoys at 60% density (10.0). The adversarial machinery of §7 was a weaker stressor than ordinary chatter, which says more about the decoys than about the models.

8. Design rules for Qontext

Three rules, each paid for with a failed run:

- **A knot must name its subject.** Entries saying “I” become unanswerable once cut from the cord; “the user works as a nurse” survives alone. This single fix took the ceiling from 6/10 to 10/10.
- **The payload is the point.** “People call me” without “Marta” is a knot tied around nothing. Never trim the name, day, place, or number out of an entry.
- **Fewer, better knots.** Density comes from selection, not truncation. Cutting entries short to meet a budget destroys exactly the meaning the budget exists to protect.

9. Limitations

Single model, single run per condition, two hand-written conversations, keyword-match scoring, and a ceiling implementation tuned by the same author who wrote the benchmark. The numbers are directional, not publication-grade. The scaffold comparison in §6 carries its own limits: one task, one configuration, and a different model from the harness runs (Shirdel-Coder-9B rather than E4B), so its result is qualitative — there is no matched pair, and an aborted run has no turn count to compare against.

10. What next

Two directions remain. First, a matched pair: re-run the harness with Shirdel-Coder-9B so the table in §6 can carry quantitative weight rather than qualitative. Second, a fair-footprint rematch: shrink Claude Code's scaffold — trim CLAUDE.md, disable unused tools — until the opening prompt fits the window, and report turns-completed-before-abort. The third direction this study always aimed at — wiring the

memory into a live agent and measuring it with a model in the loop — is done, and is §7. What that section opens up instead is the cost curve: these numbers are from a 26-turn conversation, where the saving is real but small. The claim that matters is that a pack stays constant while a transcript grows, and measuring it at 200 and 500 turns would establish that directly.

Appendix A: artifacts

quipu-experiment/ — harness.py (agent loop), task_prompt.md, acceptance_test.py, quipu-ai.py (bitstring layer, canonical Huffman), e4b_solution.py (the model's passing run-3 code), qontext-bench/ (benchmark.py, claude_memory.py ceiling), qontext-run/ (run-4 harness, bench_test.py with held-out set), qontext-live/ (live_agent.py, qontext_memory.py — the ceiling implementation as a live memory layer), claude-code-run/ (the scaffold comparison: strip_gguf.py, the reconstructed Gemma templates, shirdel-fixed.jinja, CLAUDE.md, run scripts), and run logs for all four runs.

Appendix B: packaging autopsy

The scaffold comparison consumed far more effort in setup than in experimentation. That imbalance is worth documenting, because it is invisible in a results table and is a real cost of working with locally-packaged models.

B.1 The coder GGUF was broken

The intended model was gemma4-e4b-claude-coder, distributed as an Ollama package. It had three independent defects.

- **Bundled vision and audio tensors.** The package shipped multimodal projector weights irrelevant to a text task, inflating both the file and the load. Worked around with strip_gguf.py (in claude-code-run/), which removes the unused tensors and rewrites the file.
- **No chat template.** The GGUF carried no embedded template, so llama.cpp had nothing to format conversations with. We reconstructed candidates (gemma4-official.jinja, gemma4-interleaved.jinja) from the upstream model card.
- **A mangled tokenizer.** The decisive defect. Token 106 had been renamed, so the turn markers the template emits tokenized as ordinary text rather than as control tokens. The model therefore never saw a turn boundary. This is not fixable from outside the file: the vocabulary itself is wrong.

Verdict: unusable. After stripping and re-templating, what remained was stock Gemma weights plus damage — no coder capability that plain Gemma lacks, and a tokenizer that no longer matches its own template. Abandoned.

B.2 The replacement, and its own problem

We switched to Shirdel-Coder-9B-Claude-Fable-5.Q6_K.gguf. Healthy tokenizer; tool calling verified directly against the server (a raw curl to /v1/chat/completions returns a well-formed tool_use block).

Its embedded chat template, however, rejected system messages appearing mid-conversation — which Claude Code emits routinely. Fixed by patching the template to accept them: claude-code-run/shirdel-fixed.jinja, verified to render a mid-conversation system message without error.

B.3 Working server invocation

```
./llama-b10107/llama-server \
  -m Shirdel-Coder-9B-Claude-Fable-5.Q6_K.gguf \
  --host 127.0.0.1 --port 8080 -ngl 99 -c 65536 \
  --cache-type-k q8_0 --cache-type-v q8_0 \
  --jinja --chat-template-file shirdel-fixed.jinja
```

--jinja with an explicit --chat-template-file is load-bearing: without it llama.cpp falls back to the embedded template and the mid-conversation system message fails. K/V cache quantisation to q8_0 is what makes a 65K window fit in VRAM at all.

B.4 The lesson

Of the two questions this study asks, neither is about packaging — yet packaging is where most of the wall-clock time went. A quantised GGUF is not a model; it is a model, a tokenizer, a chat template and a set of tensor conventions, any of which can be silently wrong in a way that surfaces only as bad output several layers downstream. The tokenizer defect in B.1 would have read, to anyone not looking for it, as “the small model is bad at following turn structure” — a plausible and completely false conclusion.

For anyone reproducing this work: verify the template renders, verify the special tokens survive a round-trip through the tokenizer, and verify tool calling with a raw request, before attributing any behaviour to the model.